Smart Home Automation System

EE108 Assignment — 03 by Team 04

18 December 2024

OUR TEAM

Md Rayhan Mia
Hardware Assembly / Code

Ales Vittek
TinkerCad / Code

Ava Brennan
Code

Kris. Vitkauskas
Code

Dharan Kumar
Hardware Assembly / Code

PROJECT OVERVIEW

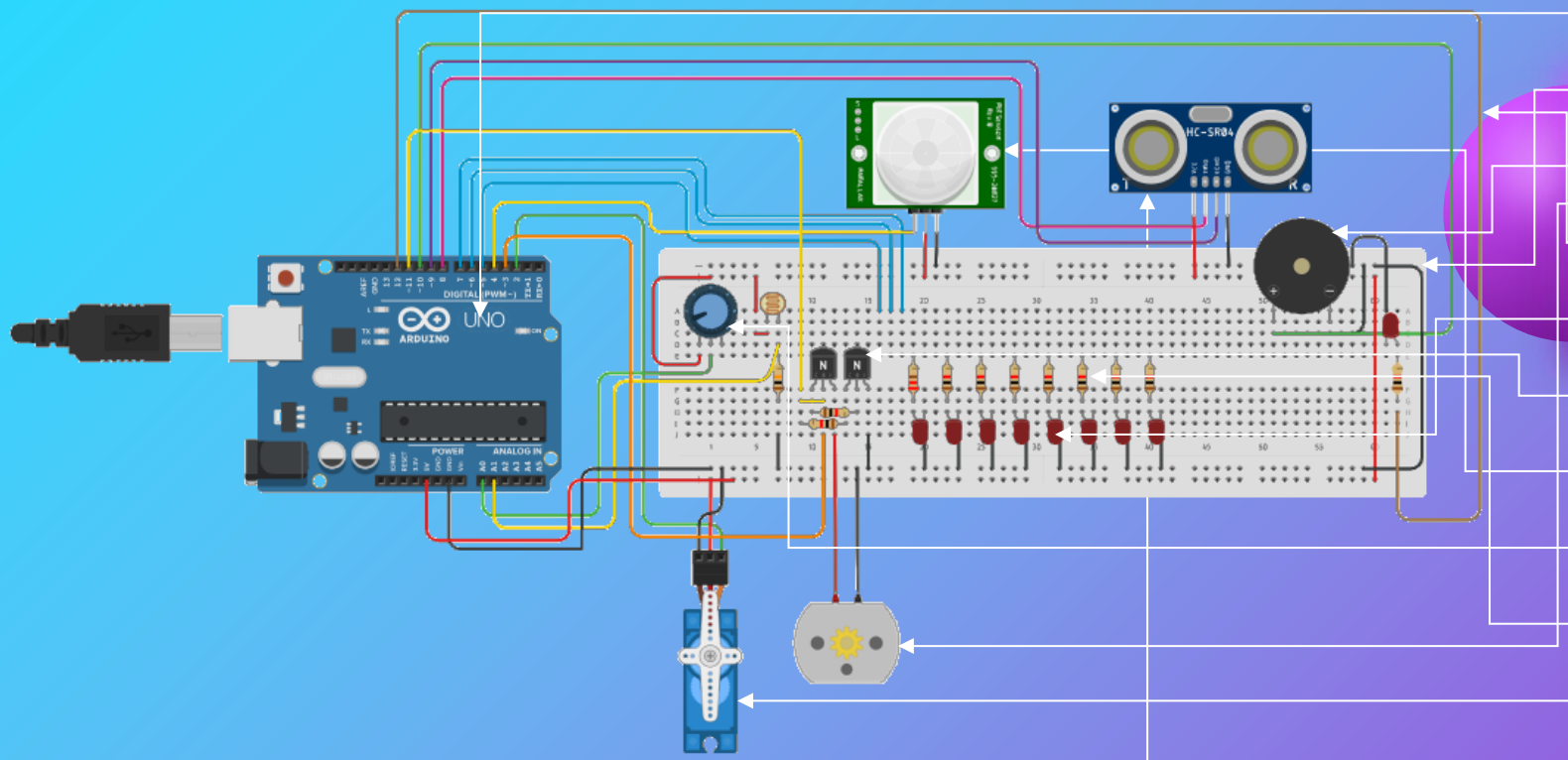
Objective

To create a smart home automation system that enhances convenience, safety, and energy efficiency by automating tasks such as lighting, curtain control, water level management, and door monitoring using sensors and actuators.

Key Features

- Automated Curtains
- Motion-Activated Lights
- Water Level Monitoring
- Sound-Activated Lighting
- Door Alerts.
- Energy Efficiency

TINKERCAD DESIGN



Components Used

- **Arduino Uno:** Acts as the main controller for the system.
- **Breadboard:** Connects all components without soldering.
- **Buzzer:** Produces sound alerts.
- **DC Motor:** Drives a mechanical load.
- **Jumper Wires:** Enables electrical connections between components.
- **LEDs:** Visual indicators for system operations.
- **NPN Transistor(s):** Switches and controls power to components.
- **PIR Sensor:** Detects motion to trigger outputs.
- **Potentiometer:** Adjusts values to test or control parameters.
- **Resistors:** Protect LEDs and components by limiting current.
- **Servo Motor:** Controls mechanical movements like opening/closing.
- **Ultrasonic Sensor (HC-SR04):** Measures distance for door status detection.

```
#include <Servo.h>  // Include the Servo library
#define SERVO_MOTOR 2          // Servo motor for controlling curtains
#define CURTAIN_MOTOR_UP 3      // Motor pin for raising the curtains
#define CURTAIN_MOTOR_DOWN 11   // Motor pin for lowering the curtains
#define IR_SENSOR 4             // IR sensor for motion detection
#define LIGHTS1 5               // Pin for controlling light 1
#define LIGHTS2 6               // Pin for controlling light 2
#define LIGHTS3 7               // Pin for controlling light 3
#define USS_TRIG 8              // Trigger pin for ultrasonic sensor
#define USS_ECHO 9              // Echo pin for ultrasonic sensor
#define BUZZER_PIN 10           // Pin for controlling the buzzer
#define LDR_PIN A0              // Pin for Light Dependent Resistor (LDR)
#define WATER_SENSOR A1         // Pin for water level sensor
#define LIGHT_PIN 12            // Pin for controlling a general light
#define SOUNDS_SENSOR_PIN A3    // Pin for sound sensor
#define WATER_THRESHOLD 100     // Threshold for water level sensor
#define LIGHT_THRESHOLD 300     // Threshold for light intensity
#define CLOSED_DOOR 5           // Threshold for door distance)
#define SOUND_THRESHOLD 300     // Threshold for sound sensor
bool curtainsup = true;        // Variable to keep track of curtain Pos
bool lightState = false;       // Variable to track light state(On/off)
Servo myServo;                 // Create a Servo object for controlling
void setup(){
  pinMode(CURTAIN_MOTOR_UP,OUTPUT); // Set pin modes for in and outputs
  pinMode(CURTAIN_MOTOR_DOWN,OUTPUT);
  pinMode(IR_SENSOR,INPUT);
  pinMode(LIGHTS1,OUTPUT);
  pinMode(LIGHTS2,OUTPUT);
  pinMode(LIGHTS3,OUTPUT);
  pinMode(USS_TRIG,OUTPUT);
  pinMode(USS_ECHO,INPUT);
  pinMode(BUZZER_PIN,OUTPUT);
  pinMode(LIGHT_PIN, OUTPUT);
  Serial.begin(9600);          // Start serial communication for debugging
  myServo.attach(SERVO_MOTOR); // Attach the servo motor to its pin
}
```

Code Part 1 – Definitions and Setup

- **Includes and defines:** Defines libraries and pin constants for motors, sensors, and lights.
- **Thresholds:** Sets sensor thresholds and initializes state flags for curtains and lights.
- **Setup Function:** Configures pin modes, starts serial communication, and attaches the servo motor.

Code Part 2 – Water Level Sensor Control

- WaterSensor: Monitors water levels using an water sensor (used potentiometer as an alternative in TinkerCad).
- Threshold: Checks if water level exceeds or drops below the set value.
- Action: Moves servo to 0 if above threshold, or to 60 if below.

```
void waterSensor(){
  // Read the water level from the water sensor
  int waterLevel = analogRead(WATER_SENSOR);

  // Check if the water level is greater than the defined
threshold
  if(waterLevel > WATER_THRESHOLD){
    // If water level is too high, close the valve (servo
to position 0)
    myServo.write(0);
  }
  else{
    // If water level is below threshold, open the valve
(servo to position 60)
    myServo.write(60);
  }
}
```

Code Part 3 – Light Control and Movement Detection

```
void lights(){
  bool movement = digitalRead(IR_SENSOR);  // Read IR sensor for motion
  int data[3] = {0, 0, 0};  // Array to store light states

  if(movement == HIGH){  // If movement detected
    for (int i = 0; i < 8; i++) {  // Loop for 8 light patterns
      int arrayPosition = 0;

      for (int j = 4; j >= 1; j = j / 2) {  // Check each bit of 'i'
        if ((i & j) > 0) {
          data[arrayPosition] = 1;  // Light ON
        } else {
          data[arrayPosition] = 0;  // Light OFF
        }
        arrayPosition++;
      }

      digitalWrite(LIGHTS1, data[0]);  // Set LIGHTS1 state
      digitalWrite(LIGHTS2, data[1]);  // Set LIGHTS2 state
      digitalWrite(LIGHTS3, data[2]);  // Set LIGHTS3 state
      delay(500);  // Wait for 500ms
    }
  }
  else{  // If no movement
    digitalWrite(LIGHTS1, LOW);  // Turn off LIGHTS1
    digitalWrite(LIGHTS2, LOW);  // Turn off LIGHTS2
    digitalWrite(LIGHTS3, LOW);  // Turn off LIGHTS3
  }
}
```

- **Motion Sensor:** Detects motion using an PIR sensor.
- **Action:** Activates lights in binary patterns when motion is detected.
- **No Motion:** Turns lights off if no motion is sensed.

Code Part 4 – Curtain Control Based on Light Levels

- **LightSensor:** Reads light levels with an LDR (Photoresistor) sensor.
- **Threshold:** Compares light levels to the set value.
- **Action:** Raises curtains if levels exceed the threshold and they are down, lowers them if levels drop and they are up.

```
void curtains(bool &curtainsup){
  int lightLevel = analogRead(LDR_PIN);  // Read light level
from LDR

  if (lightLevel > LIGHT_THRESHOLD && !curtainsup){  // If
light level is high and curtains are up
    digitalWrite(CURTAIN_MOTOR_UP, HIGH);  // Move curtains up
    delay(400);  // Wait for 400ms to allow curtains to move
    digitalWrite(CURTAIN_MOTOR_UP, LOW);  // Stop motor
    curtainsup = true;  // Mark curtains as up
  }

  if (lightLevel < LIGHT_THRESHOLD && curtainsup){  // If light
level is low and curtains are down
    digitalWrite(CURTAIN_MOTOR_DOWN, HIGH);  // Move curtains
down
    delay(400);  // Wait for 400ms to allow curtains to move
    digitalWrite(CURTAIN_MOTOR_DOWN, LOW);  // Stop motor
    curtainsup = false;  // Mark curtains as down
  }
}
```

Code Part 5 – Door Sensor and Reading Distance

```
void openedDoor(){
  if(readSensor() > CLOSED_DOOR){  // If door is open (distance > threshold)
    digitalWrite(BUZZER_PIN, HIGH);  // Activate the buzzer
  }
  else{
    digitalWrite(BUZZER_PIN, LOW);  // Turn off the buzzer if the door is closed
  }
}

int readSensor(){
  long duration = 0;
  int distance = 0;

  digitalWrite(USS_TRIG, LOW);  // Set the trigger pin low to start measurement
  delayMicroseconds(2);  // Wait for 2 microseconds

  digitalWrite(USS_TRIG, HIGH);  // Send a pulse to the ultrasonic sensor
  delayMicroseconds(10);  // Wait for 10 microseconds
  digitalWrite(USS_TRIG, LOW);  // Stop the pulse

  duration = pulseIn(USS_ECHO, HIGH);  // Measure the duration of the echo pulse

  distance = 0.017 * duration;  // Calculate the distance based on duration
  Serial.print(distance);  // Print distance for debugging
  return distance;  // Return the measured distance
}
```

- **Detects Door Status:** Uses ultrasonic sensor to determine if the door is open or closed.
- **Measures Distance:** Sends a pulse, measures the echo, and calculates the distance.
- **Controls Buzzer:** Activates the buzzer when the door is open and deactivates when closed.

Code Part 6 – Sound Trigger Lights and Main Loop

- **Sound-Based Control:** Toggles lights based on sound sensor input.
- **Main Loop**: Continuously checks sensors and triggers actions like curtain movement and buzzer.
- **Sensor Integration:** Uses sensor data for real-time control of the environment.

```
void soundLights() {
  int soundLevel = analogRead(SOUNDS_SENSOR_PIN);  // Read the sound sensor

  if (soundLevel > SOUND_THRESHOLD) {  // If sound level exceeds the threshold
    delay(200);  // Delay to debounce the sound trigger
    lightState = !lightState;  // Toggle the light state (ON/OFF)
    digitalWrite(LIGHT_PIN, lightState ? HIGH : LOW);  // Set the light accordingly
  }
}

void loop(){
  waterSensor();  // Check the water sensor
  lights();  // Control lights based on motion
  curtains(curtainsup);  // Control curtains based on light level
  openedDoor();  // Check if the door is open and trigger the buzzer
  soundLights();  // Trigger lights based on sound sensor
}
```

CONCLUSION

Key Learnings

- ✓ Room Climate Monitoring: Measures temperature, humidity, and pressure, displaying data on an LCD with LED indicators.
- ✓ User-Friendly Interface: Keypad allows users to select which reading to view easily.
- ✓ Real-Time Alerts: LEDs indicate if conditions are too high, normal, or too low.

Future Improvements

- ❑ Smart Home Automation: Integrate with HVAC for automatic climate control.
- ❑ Greenhouses and Labs: Monitor and maintain optimal environmental conditions.
- ❑ Warehouses: Track conditions to protect sensitive stored items.